

Week 7 Lab Worksheet

The objective of this Week 7 laboratory session was to evaluate the evolution of network security from traditional perimeter-based models to modern Zero Trust architectures. We began by architecting a Site-to-Site GRE/IPsec tunnel to secure network-layer traffic across an untrusted medium (LO1). Transitioning to the cloud, we analysed TLSv1.3 session-based VPN models, capturing the cryptographic handshakes that establish secure transport-layer tunnels (LO2). Finally, we implemented and evaluated a Zero Trust Network Access (ZTNA) framework, demonstrating how identity and device posture dictate per-request authorization, and critically analysing the limitations this places on traditional network traffic inspection (LO3 & LO4).

Learning Objectives

1. Architect and emulate secure Site-to-Site network layer tunnels utilizing encapsulation and cryptographic protocols.
2. Analyse transport-layer security mechanisms and session-based authentication models over public cloud infrastructure.
3. Evaluate Zero Trust Network Access (ZTNA) paradigms by contrasting session-based perimeter security with identity- and posture-based micro-segmentation.
4. Utilise packet analysis tools to critically assess the limitations of network-level inspection in encrypted, application-layer traffic.

1.Site-to-Site VPN with GRE + IPsec within Mininet Hosts

Topology is as below

remote --- s1 --- remote_gw ---+ s3 (the "internet") ---+ main_gw --- s2 --- main

Remote LAN 192.168.10.0/24 Main LAN 10.0.0.0/24

Public side 203.0.113.0/24

1. Navigate to 7COM2008/LAB6
 - a. `sudo python3 lab6.py`
 - b. Try a ping between `main` and `remote`
 - i. `main ping remote`
 - c. Initiate a TCP transfer between `main` and `remote`
 - i. Refer to lab worksheet (Lab2 Week3) over instructions
 - d. Capture the traffic on `s3-eth2`.
 - i. Make notes over the traffic pattern, protocol, source and destination IP addresses

2. Cloud VPN Session Model using TLSv1.3

We are now creating a TLSv1.3 tunnel between Mininet hosts and a render cloud deployment.

- a. `sudo python3 lab6-1.py`
- b. Open wireshark on h1 host using the below command in Mininet
 - a. `h1 wireshark &`
 - b. `h1 curl -s --resolve ztna-gateway-a8js.onrender.com:443:216.24.57.18 https://ztna-gateway-a8js.onrender.com/`

You will be able to see the TLS handshake and traffic that follows.

1	0.000000000	fe80::84db:68ff:feff::2	ff02::2	ICMPv6	70 Router Solicitation from 86:db:68:f0:9f:72
2	16.202615567	10.0.0.1	216.24.57.18	TCP	74 43592 → 443 [SYN] Seq=0 Win=42340 Len=0 MSS
3	16.215827436	216.24.57.18	10.0.0.1	TCP	58 443 → 43592 [SYN, ACK] Seq=0 Ack=1 Win=6424
4	16.215853534	10.0.0.1	216.24.57.18	TCP	54 43592 → 443 [ACK] Seq=1 Ack=1 Win=42340 Len=0
5	16.218873722	10.0.0.1	216.24.57.18	TLSv1.3	571 Client Hello
6	16.219262273	216.24.57.18	10.0.0.1	TCP	54 443 → 43592 [ACK] Seq=1 Ack=518 Win=64240 Len=0
7	16.251783129	216.24.57.18	10.0.0.1	TLSv1.3	1506 Server Hello, Change Cipher Spec
8	16.251792447	10.0.0.1	216.24.57.18	TCP	54 43592 → 443 [ACK] Seq=518 Ack=1453 Win=4216
9	16.252933112	216.24.57.18	10.0.0.1	TLSv1.3	1399 Application Data
10	16.252941245	10.0.0.1	216.24.57.18	TCP	54 43592 → 443 [ACK] Seq=518 Ack=2798 Win=4216
11	16.253725388	10.0.0.1	216.24.57.18	TLSv1.3	134 Change Cipher Spec, Application Data
12	16.253817844	216.24.57.18	10.0.0.1	TCP	54 443 → 43592 [ACK] Seq=2798 Ack=598 Win=6424
13	16.253827663	10.0.0.1	216.24.57.18	TLSv1.3	100 Application Data

3. Zero Trust Network Access (ZTNA)

Unlike TLSv1.3 access is decided per request against a policy table on the server:

Identity	Allowed resources	Required posture
alice	finance-app	compliant
bob	devtools, hr-app	compliant

- a. `sudo python3 lab6-2.py`
- b. Open wireshark and h1 using the below command in Mininet and observe traffic
 - a. `h1 wireshark &`
 - b. Let's try Zero Trust model with an allowed request and deny request
 - i. Allow request (copy the below)
 1. `h1 curl -s --resolve ztna-gateway-a8js.onrender.com:443:216.24.57.18 -X POST -H "Content-Type: application/json" -d '{"identity":"alice","device_posture":"compliant","resource":"finance-app"}' https://ztna-gateway-a8js.onrender.com/ztna/authorize`
 2. Observe the traffic in wireshark
 - ii. Deny Request
 1. `h1 curl -s --resolve ztna-gateway-a8js.onrender.com:443:216.24.57.18 -X POST -H "Content-Type: application/json" -d '{"identity":"alice","device_posture":"compliant","resource":"hr-app"}' https://ztna-gateway-a8js.onrender.com/ztna/authorize`
 2. Observe the traffic in wireshark

Both are the same shape of request, same TLS session, same endpoint. Only the resource field changes, and that's the entire point for the class: identical "shape" of traffic on the wire either way, so Wireshark alone can't tell allow from deny, only the decrypted JSON body shows it.